

Computer Vision - 2026

Week #04. Generative models

Lectures by Alexei Kornaev ^{1,2}

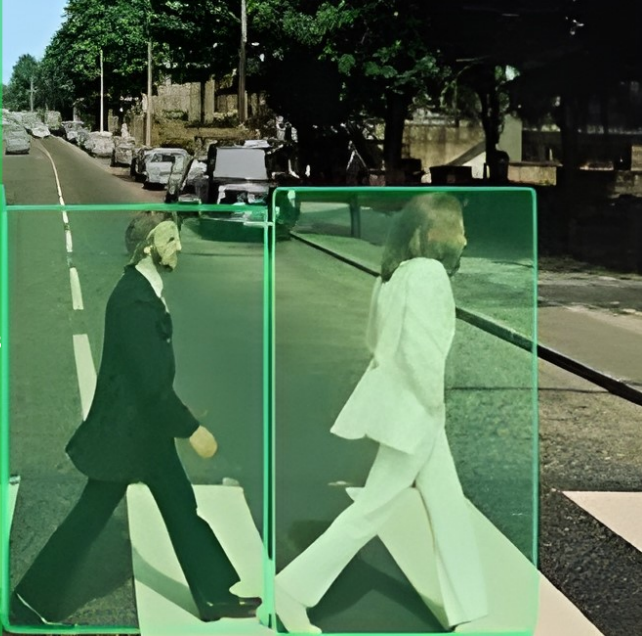
Practical sessions by Kirill Yakovlev ^{1,3}

¹The Center for Top-Level Educational Programs
in AI, Innopolis University (IU), Innopolis

²RC for AI, National RC for Oncology
n.a. N.N.Blokhin, Moscow

³Ak Bars Bank, Kazan

February 9, 2026



Agenda

CV-2026

A.Kor-naev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

① Outcomes

② Recap (Probability Basics)

③ Variational Autoencoders (VAE) Diederik and Max [2019]

④ Diffusion Models

⑤ Flow Matching (FM)

CV-2026

A.Korinaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Section 1. Outcomes

Outcomes

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

This week's lecture builds an operational understanding of **generative modeling** as **learning distributions** and **sampling-based inference/planning**. By the end of the week, students will be able to:

- 1 Distinguish **discriminative** vs **generative** modeling:

$$p(y | x) \quad \text{vs} \quad p(x), \quad p(x | c), \quad p(x, y).$$

- 2 Explain the **VAE** pipeline: encoder $q_{\phi}(z | x)$, decoder $p_{\theta}(x | z)$, and the **ELBO** decomposition (reconstruction + KL).
- 3 Explain **diffusion models** conceptually as iterative denoising: forward noise schedule $x_0 \rightarrow x_t$ and learned reverse process (noise prediction).
- 4 Recognize **flow/flow-matching** at a high level: distribution transport via an invertible map or a learned vector field.
- 5 Connect generative models to robotics: **policy as a generative model** and/or **planning via sampling**.

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Section 2. Recap (Probability Basics)

Recap: Random Variables and Distributions

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Objects

- x — data random variable (image, feature vector, trajectory).
- c — observed condition (class, text prompt, state, goal).
- θ — model parameters (NN weights), fixed during inference.

Distributions

- Discrete: $p(x)$ is a probability mass function (pmf).
- Continuous: $p(x)$ is a probability density function (pdf); probabilities are integrals:

$$\mathbb{P}(x \in A) = \int_A p(x) dx.$$

Recap: Conditional Probability and Bayes Rule

CV-2026

A.Kor-naev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Conditionals

$$p(x | c) = \frac{p(x, c)}{p(c)} \quad (\text{when } p(c) > 0).$$

Bayes rule (what we actually use)

$$p(c | x) = \frac{p(x | c)p(c)}{p(x)}, \quad p(x) = \sum_c p(x | c)p(c) \text{ or } \int p(x | c)p(c) dc.$$

Bridge: Generative modeling often specifies $p_\theta(x | c)$ and uses sampling/inference downstream.

Recap: Expectation and Monte Carlo

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Expectation

$$\mathbb{E}_{x \sim p}[f(x)] = \int f(x) p(x) dx \quad (\text{or } \sum_x f(x)p(x)).$$

Monte Carlo estimator (sampling)

If $x_i \sim p(x)$ i.i.d., then

$$\mathbb{E}_{x \sim p}[f(x)] \approx \frac{1}{N} \sum_{i=1}^N f(x_i).$$

Why it matters: Training and sampling procedures in VAEs / diffusion / flows rely on Monte Carlo estimates.

Recap: Likelihood and Maximum Likelihood Estimation (MLE)

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Model family

A generative model defines a family of densities indexed by parameters θ :

$$p_{\theta}(x), \quad p_{\theta}(x \mid c).$$

Log-likelihood objective

Given data $\{x^{(i)}\}_{i=1}^n$, MLE trains by

$$\max_{\theta} \sum_{i=1}^n \log p_{\theta}(x^{(i)}).$$

Notation tip: “density” vs “likelihood” is the same function viewed as $p_{\theta}(\cdot)$ in x or in θ (with x fixed).

Recap: Cross-Entropy and KL Divergence

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

KL divergence

$$\text{KL}(q\|p) = \mathbb{E}_{z \sim q} [\log q(z) - \log p(z)] \geq 0.$$

Cross-entropy (classification link)

For labels y with true distribution $p(y)$ and model $q_\theta(y)$:

$$H(p, q_\theta) = \mathbb{E}_{y \sim p} [-\log q_\theta(y)].$$

Minimizing cross-entropy $\Leftrightarrow$ maximizing likelihood.

Bridge: In VAEs, a KL term regularizes an approximate posterior toward a prior.

Recap: Latent Variables and Marginalization

CV-2026

A.Korbaev,
K.Yakovlev

Latent-variable model

Introduce latent z (unobserved):

$$p_{\theta}(x, z) = p(z) p_{\theta}(x | z).$$

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Marginal likelihood

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p(z) p_{\theta}(x | z) dz.$$

- This integral is the source of difficulty in training latent-variable models.
- VAEs solve it by optimizing a lower bound (ELBO).

Recap: Datasets and Practical Targets

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Common datasets for generative modeling labs

- **MNIST / Fashion-MNIST:** fast training, stable reconstructions, clean interpolation.
- **CelebA:** better visuals, more compute.
- **CIFAR-10:** harder; simple VAEs often produce weak samples.

Training target (generic)

Most generative training reduces to minimizing a negative log-likelihood:

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)],$$

or its tractable surrogate (ELBO / denoising regression / flow matching regression).

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Section 3. Variational Autoencoders (VAE) Diederik and Max [2019]

AE vs VAE (intuition)

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

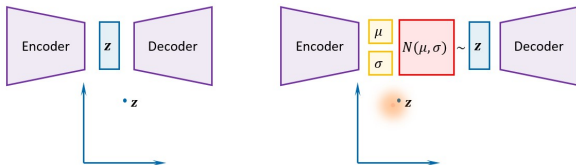
Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)



Deterministic AE learns a code for reconstruction;
VAE learns a **distribution** in latent space enabling
sampling.

What to look at

- **AE:** $x \rightarrow z \rightarrow \hat{x}$ (point code)
- **VAE:**
 $x \rightarrow (\mu, \sigma) \rightarrow z \rightarrow \hat{x}$
- Prior $p(z) = \mathcal{N}(0, I)$ makes the latent space **sampleable**

Video (20 min)

YouTube: AE vs VAE
intuition

Autoencoder vs VAE (why probabilistic?)

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Deterministic autoencoder (AE)

$$z = \text{Enc}_\phi(x), \quad \hat{x} = \text{Dec}_\theta(z)$$

Optimizes a reconstruction loss (e.g., MSE), but does not define $p_\theta(x)$.

VAE: probabilistic generative model

$$z \sim p(z) = \mathcal{N}(0, I), \quad x \sim p_\theta(x | z).$$

Encoder outputs a distribution:

$$q_\phi(z | x) \approx p_\theta(z | x).$$

Latent-variable model and the hard objective

CV-2026

A.Korbaev,
K.Yakovlev

Joint distribution

$$p_{\theta}(x, z) = p(z) p_{\theta}(x | z).$$

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

What we would like to maximize

$$\log p_{\theta}(x) = \log \int p(z) p_{\theta}(x | z) dz.$$

Diffusion
Models

Flow
Matching
(FM)

- The integral is generally intractable for expressive decoders.
- We introduce an approximate posterior $q_{\phi}(z | x)$ and optimize a lower bound.

ELBO (Evidence Lower Bound): final form

CV-2026

A.Kornaev,
K.Yakovlev

ELBO

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

$$\log p_{\theta}(x) \geq \underbrace{\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)]}_{\text{reconstruction / data fit}} - \underbrace{\text{KL}(q_{\phi}(z | x) \| p(z))}_{\text{regularization to prior}}.$$

Training objective

Maximize ELBO $\Leftrightarrow$ minimize negative ELBO:

$$\mathcal{L}_{\text{VAE}}(\theta, \phi) = -\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] + \text{KL}(q_{\phi}(z | x) \| p(z)).$$

ELBO derivation sketch

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Key identity

Start from:

$$\log p_{\theta}(x) = \log \int q_{\phi}(z | x) \frac{p_{\theta}(x, z)}{q_{\phi}(z | x)} dz$$

Apply Jensen:

$$\log p_{\theta}(x) \geq \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z | x)].$$

Rearrange

$$\mathbb{E}_q[\log p_{\theta}(x, z) - \log q] = \mathbb{E}_q[\log p_{\theta}(x | z)] - \text{KL}(q_{\phi}(z | x) \| p(z)).$$

Reparameterization trick (why backprop works)

CV-2026

A.Kornaev,
K.Yakovlev

Encoder outputs Gaussian parameters

$$q_{\phi}(z \mid x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x))).$$

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Reparameterize sampling

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Diffusion
Models

Now randomness is in ε , enabling gradients through $\mu_{\phi}, \sigma_{\phi}$.

Flow
Matching
(FM)

Key point: Sampling becomes a deterministic function of (x, ε) .

What is the reconstruction loss exactly?

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Decoder likelihood choice determines the loss

- **Bernoulli decoder** (binary pixels): $p_{\theta}(x | z) = \prod_j \text{Bernoulli}(x_j; \hat{x}_j)$

$$-\log p_{\theta}(x | z) \equiv \text{BCE}(x, \hat{x}).$$

- **Gaussian decoder** (continuous pixels): $p_{\theta}(x | z) = \mathcal{N}(x; \hat{x}, \sigma^2 I)$

$$-\log p_{\theta}(x | z) = \frac{1}{2\sigma^2} \|x - \hat{x}\|_2^2 + \text{const.}$$

Practical default: MNIST often uses BCE; natural images commonly use Gaussian or discretized likelihoods.

KL term: closed form for Gaussians

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

When $q_{\phi}(z | x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and $p(z) = \mathcal{N}(0, I)$

$$\text{KL}(q||p) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1) .$$

- Penalizes large means and variances away from the prior.
- Prevents the latent space from becoming arbitrary/unstructured.

Beta-VAE: a controllable trade-off

CV-2026

A.Kor-naev,
K.Yakovlev

Beta-VAE objective

$$\mathcal{L}_{\beta\text{-VAE}} = -\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] + \beta \cdot \text{KL}(q_{\phi}(z | x) \| p(z)).$$

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

- $\beta > 1$: stronger disentanglement pressure, often worse reconstructions.
- $\beta < 1$: better reconstructions, weaker latent regularization.

Teaching point: This is an explicit “information bottleneck” knob.

CV-2026

A.Kor-naev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Section 4. Diffusion Models

Diffusion models: the core idea Ho et al. [2020]

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

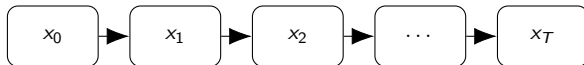
Generative modeling as **iterative denoising**

Diffusion models:

- gradually **add noise** to data (forward process),
- then learn to **remove noise step by step** (reverse process).

$$x_0 \xrightarrow{\text{add noise}} x_1 \rightarrow \dots \rightarrow x_T \approx \mathcal{N}(0, I)$$

increasing noise



Forward diffusion process (fixed, no learning)

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Add Gaussian noise with a schedule

At step t :

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

where α_t decreases with t .

Key properties

- Forward process is **known and fixed**.
- After many steps, x_T becomes almost pure noise.
- We never train the forward process.

Intuition

Destroy the structure in small steps so that learning the reverse becomes easy.

Reverse process: learning to denoise

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

What the neural network learns

Given a noisy sample x_t and time t , predict the noise:

$$\varepsilon_{\theta}(x_t, t) \approx \varepsilon$$

Why predict noise (not x_0 directly)?

- Noise has a simple, stable distribution.
- Same objective works for all t .
- Empirically more stable than predicting clean data.



predicted noise

Diffusion training algorithm (DDPM-style)

CV-2026

A.Kor-naev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Algorithm 1 Diffusion model training (noise prediction)

[1]

training steps N , batch size B

for step = 1 to N **do**

 sample clean data $x_0 \sim p_{\text{data}}(x)$

 sample time $t \sim \{1, \dots, T\}$

 sample noise $\varepsilon \sim \mathcal{N}(0, I)$

 create noisy sample $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\varepsilon$

 predict $\hat{\varepsilon} = \varepsilon_{\theta}(x_t, t)$

 update θ to minimize $\text{MSE}(\hat{\varepsilon}, \varepsilon)$

end for

Summary

Randomize t , add noise, and predict the noise.

Sampling: generate by denoising

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

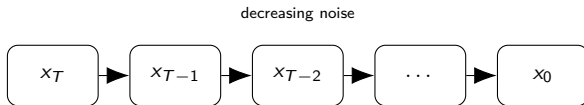
Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

After training

- Start from pure noise $x_T \sim \mathcal{N}(0, I)$.
- Iteratively remove noise using the learned model.



Practical note

Sampling is slow: tens to hundreds of denoising steps.

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Section 5. Flow Matching (FM)

Flow Matching Lipman et al. [2023]: the core idea (intuition)

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Generative modeling as **transport**

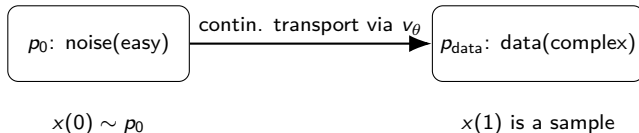
We want to transform a simple **base distribution** into the **data distribution**.

$$x_0 \sim p_0(x) \text{ (easy, e.g. } \mathcal{N}(0, I)) \implies x_1 \sim p_{\text{data}}(x)$$

Flow Matching learns a **velocity field** that moves samples continuously:

$$\frac{dx(t)}{dt} = v_{\theta}(x(t), t), \quad t \in [0, 1]$$

Sampling = draw $x(0) \sim p_0$ and integrate ODE to get $x(1)$.



FM vs Diffusion vs Normalizing Flows (comparison)

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Method	Dynamics	Training target	Sampling
Normalizing flow	invertible map $x = f_{\theta}(z)$	exact log-likelihood	one forward pass
Diffusion	stochastic SDE / denoising	predict noise/score	many denoise steps
Flow Matching	deterministic ODE $\dot{x} = v_{\theta}$	match target velocity field	ODE integration (Euler/RK)

Why FM is easy to teach

- FM looks like **physics**: a velocity field moves particles.
- Training is a standard regression: predict a velocity vector.
- Sampling is a standard numerical method: Euler / Runge–Kutta.

FM training: linear coupling (the cleanest version)

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Two endpoints define a path

- x_1 = a **real data sample** (e.g., an image, or a whole trajectory).
- x_0 = a **noise sample** of the same shape (usually Gaussian).
- Choose a simple path between them (linear):

$$x_t = (1 - t)x_0 + tx_1, \quad t \in [0, 1]$$

Key observation (target velocity)

Differentiate the linear path:

$$\frac{dx_t}{dt} = x_1 - x_0 \quad \Rightarrow \quad v^*(x_t, t) = x_1 - x_0$$

Flow Matching trains $v_\theta(x, t)$ to predict this velocity at random (x_t, t) pairs.

FM algorithm (training only)

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Algorithm 2 Flow Matching (FM) training with linear coupling

[1]

training steps N , batch size B

for step = 1 to N **do**

 sample data batch $x_1 \sim p_{\text{data}}(x)$

▷ images or trajectories

 sample noise batch $x_0 \sim \mathcal{N}(0, I)$

▷ same shape as x_1

 sample random times $t \sim U(0, 1)$

▷ independent per item

 build interpolates $x_t \leftarrow (1 - t)x_0 + tx_1$

 set target velocity $v^* \leftarrow x_1 - x_0$

 predict $\hat{v} \leftarrow v_\theta(x_t, t)$

 update θ to minimize $\text{MSE}(\hat{v}, v^*)$

end for

What students should remember

Sampling: ODE integration (Euler vs RK)

CV-2026

A.Korbaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

After training

Start from noise and integrate the learned ODE: $x(0) \sim \mathcal{N}(0, I)$, $\dot{x} = v_\theta(x, t)$, $t : 0 \rightarrow 1$

Euler (simple)

$$x \leftarrow x + \Delta t \cdot v_\theta(x, t)$$

Pros: minimal code.

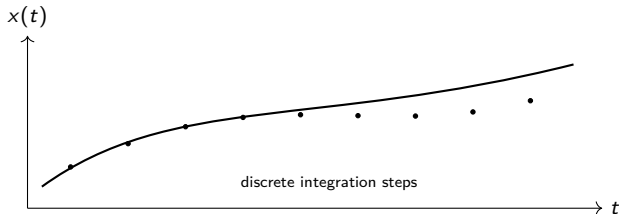
Cons: can be unstable if steps are too large.

Runge–Kutta (RK4, stable)

Uses 4 evaluations of v_θ per step.

Pros: smoother trajectories for same step count.

Cons: $\approx 4\times$ slower than Euler.



Trajectory planning view (robotics-friendly explanation)

CV-2026

A.Korinaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

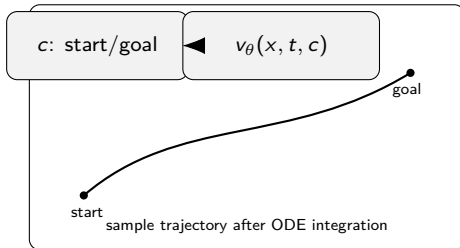
Flow
Matching
(FM)

Interpretation for trajectories

Let x be the **whole trajectory** (all waypoints concatenated): $x \in \mathbb{R}^{2T}$ (2D path of length T)
FM learns to generate trajectories by transporting noise trajectories to “expert-like” ones.

Why conditional FM is useful (optional, but practical)

Condition c can be start/goal, obstacle map, or image features: $\dot{x} = v_{\theta}(x, t, c)$ Then the generator produces **task-specific** trajectories.



Hands-on: Flow Matching (FM) for Trajectory Planning

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Algorithm 3 Flow Matching (FM) Training (linear coupling)

[1]

training steps N , batch size B

▷ N = number of gradient updates

for step = 1 to N **do**

▷ one mini-batch update

 Sample real trajectories $x_1 \sim p_{\text{data}}(x \mid c)$ and (optionally) conditions c

▷ In our hands-on: $c = [s, g]$

 Sample noise trajectories $x_0 \sim \mathcal{N}(0, I)$

▷ same shape as x_1

 Sample random time $t \sim \mathcal{U}(0, 1)$

▷ yes, random t every batch/item

 Build an interpolated point on the path: $x_t \leftarrow (1 - t)x_0 + tx_1$

▷ A simple transport path

 Define target velocity for this coupling: $v^* \leftarrow x_1 - x_0$

▷ For linear coupling, $\frac{d}{dt}x_t = x_1 - x_0$ (constant in t)

 Predict velocity: $\hat{v} \leftarrow v_\theta(x_t, t, c)$

▷ time input is required

 Update parameters θ by minimizing $\mathcal{L}_{FM} = \text{MSE}(\hat{v}, v^*)$

end for

Image generation view (how to connect to diffusion intuition)

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Same FM pipeline, different object

For images, x is a tensor (pixels/latents). FM still learns $\dot{x} = v_\theta(x, t)$.

Diffusion vs FM

- Diffusion: learn to **denoise** (stochastic reverse process).
- FM: learn a **deterministic drift** that moves samples to data.
- Both use time $t \in [0, 1]$ and many small update steps at sampling.

Question	Diffusion answer	FM answer
What is predicted?	noise / score	velocity (drift)
Sampling	denoise steps	ODE steps
Deterministic?	usually no	yes (given noise seed)

Some of the failure modes

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)

Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

Typical issues in our **hands-on** session

- **Shaky trajectories:** model treats trajectory as a flat vector, no local coupling.
- **Endpoint drift:** discretized integration misses exact start/goal.
- **Too few sampling steps:** Euler with large Δt gives artifacts.

Practical fixes (minimal)

- Use a **sequence-aware** velocity field (Conv1D / Transformer over time index).
- Add a small **smoothness penalty** on predicted velocity across time-steps.
- Add an **endpoint penalty** (start/end match) and/or use RK4 in sampling.

Key message

FM is simple: **randomize t + regress a velocity**. Quality depends on **architecture** and **numerical integration**.

Bibliography

CV-2026

A.Kornaev,
K.Yakovlev

Outcomes

Recap
(Probability
Basics)

Variational
Autoencoders
(VAE)
Diederik and
Max [2019]

Diffusion
Models

Flow
Matching
(FM)

P. Kingma Diederik and Welling Max. An introduction to variational autoencoders. **Foundations and Trends® in Machine Learning**, 12(4):307â392, November 2019. ISSN 1935-8245. doi: 10.1561/22000000056. URL <http://dx.doi.org/10.1561/22000000056>.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. URL <https://arxiv.org/abs/2210.02747>.